

LAB 5 ASSIGNMENT

Docker & Containerization – Dockerizing the Lab 4 Student REST API

Web Services and SOA

1. Objective

In this lab, you will learn Docker by containerizing the Student REST API developed in Lab 4. You will work step by step to install and verify Docker, create a Docker image, run the API inside a container, test the containerized REST API using Postman, connect the API container to a MongoDB container through a Docker network, configure the connection using environment variables, persist MongoDB data using a Docker volume, and finally run the complete application using Docker Compose.

2. Lab Context

This lab continues the application developed in the previous labs:

- Lab 1: Created the CampusConnect frontend.
- Lab 2: Integrated REST APIs into the frontend.
- Lab 3: Developed RESTful web services.
- Lab 4: Connected clients, backend services, and MongoDB persistence.
- Lab 5: Dockerize the Lab 4 Student REST API and its database environment.

3. Install and Verify Docker

Complete this section before starting the Dockerization steps. Use the official Docker installation instructions for the current operating-system requirements.

3.1 Windows – Docker Desktop

1. Download and install Docker Desktop for Windows from the official Docker website.
2. Run the installer and keep the recommended options.
3. If prompted, enable or install the required WSL 2 components.
4. Restart the system if requested.
5. Open Docker Desktop and wait until Docker is running.
6. Open Command Prompt or PowerShell.

```
docker --version
```

```
docker run hello-world
```

If the version command displays the Docker version and hello-world completes successfully, Docker is ready.

3.2 Linux – Docker Engine

7. Install Docker Engine using the official installation instructions for your Linux distribution.
8. Open a terminal after installation.
9. Verify the Docker installation.

10. Run the hello-world container to confirm that Docker can create and run containers.

```
docker --version  
docker run hello-world
```

4. Step-by-Step Practical

Follow the steps in order. Do not move to the next step until the verification point of the current step is successful.

Step 1 – Prepare and Verify the Lab 4 Student REST API

Use the Student REST API that you developed in Lab 4. First verify that it works normally without Docker.

11. Start MongoDB using the setup currently used by your Lab 4 project.
12. Start the Student REST API using your normal development command.
13. Confirm that the API starts successfully and note the port on which it is running.
14. Use Postman to call at least one existing Student API endpoint.
15. Confirm that the expected response and HTTP status code are received.

Typical Student REST API endpoints include:

- GET /students
- GET /students/{id}
- POST /students
- PUT /students/{id}
- DELETE /students/{id}

Verification: The Student REST API must work correctly before Dockerization begins.

Step 2 – Create the Dockerfile

Create a file named Dockerfile in the appropriate backend/project directory. Modify the example according to your Lab 4 technology stack.

- Select an appropriate base image.
- Set a working directory.
- Copy the dependency files.
- Install the required dependencies.
- Copy the application source code.
- Expose the API port.
- Define the command required to start the application.

```
FROM node:20  
WORKDIR /app  
COPY package*.json ./  
RUN npm install  
COPY . .  
EXPOSE 3000  
CMD ["npm", "start"]
```

Verification: Dockerfile exists and contains all required instructions for building the Student API image.

Step 3 – Build the Student API Docker Image

Open a terminal in the directory containing the Dockerfile and build the image.

```
docker build -t student-api:v1 .
```

```
docker images
```

Verification: Confirm that student-api:v1 appears in the list of local Docker images.

Step 4 – Run the Student API Container

Run the image as a container and map the container port to the host machine.

```
docker run --name student-api -p 3000:3000 student-api:v1
```

The first port is the host port and the second port is the container port. Replace 3000 with the actual port used by your application if necessary.

In another terminal, verify the running container:

```
docker ps
```

Verification: The student-api container should be running and the API port should be mapped.

Step 5 – Test the Containerized Student REST API with Postman

Send requests from Postman to the host port mapped to the API container. Demonstrate at least three requests from your existing Lab 4 API.

- GET /students
- GET /students/{id}
- POST /students
- PUT /students/{id}
- DELETE /students/{id}

For each demonstrated request, verify:

- The request reaches the containerized API.
- The expected response is returned.
- The appropriate HTTP status code is received.
- Data is correctly retrieved or modified where applicable.

Verification: Your REST API must work when it is running inside the Docker container.

Step 6 – Run MongoDB as a Docker Container

Now move from a single container to a multi-container application. Run MongoDB in a separate container so that the Student API and database can communicate through Docker networking.

```
docker pull mongo
```

```
docker run -d --name mongodb mongo
```

```
docker ps
```

Verification: Confirm that both the Student API and MongoDB containers are running.

Step 7 – Create a Docker Network and Connect the Containers

Create a dedicated Docker network for the application:

```
docker network create student-network
```

Connect both containers to the network:

```
docker network connect student-network mongodb
```

```
docker network connect student-network student-api
```

The API must communicate with MongoDB using the MongoDB container/service name rather than localhost. For example, the MongoDB host can be configured as mongodb and the standard MongoDB port is 27017.

Concept:

```
student-api container → mongodb:27017 → mongodb container
```

Do not use localhost:27017 from inside the Student API container to refer to the MongoDB container.

Verification: The Student API should be able to reach MongoDB through the Docker network.

Step 8 – Configure Environment Variables

Configure the Student API so that environment-specific values are not hard-coded. At minimum, demonstrate the API port and MongoDB connection configuration.

```
PORT=3000
```

```
MONGO_URI=mongodb://mongodb:27017/campusconnect
```

Use the environment-variable mechanism supported by your technology stack. The exact database name may be changed to match your Lab 4 application.

Verification: The API starts using the supplied environment variables and connects to the MongoDB container.

Step 9 – Add a MongoDB Volume and Verify Persistence

Use a Docker volume so that MongoDB data is stored outside the temporary writable layer of the MongoDB container.

```
docker volume create student-mongo-data
```

When running MongoDB, mount the volume to MongoDB's data directory. For the official MongoDB image, the data directory is commonly /data/db.

```
docker run -d --name mongodb -v student-mongo-data:/data/db mongo
```

Persistence exercise:

16. Insert a Student record through the Student REST API.
17. Confirm that the record can be retrieved.
18. Stop and remove the MongoDB container.
19. Create a new MongoDB container using the same volume.
20. Start/connect the Student API again if required.
21. Retrieve the Student record and verify that it is still present.

Verification: Data should remain available after the MongoDB container is recreated because the data is stored in the volume.

Step 10 – Create Docker Compose for the Student Application

Create compose.yaml (or docker-compose.yaml) to define the Student API and MongoDB as application services. The Compose configuration should also define the network and MongoDB volume required by the application.

```
services:
  api:
    build: .
```

```
container_name: student-api
ports:
  - "3000:3000"
environment:
  PORT: 3000
  MONGO_URI: mongodb://mongodb:27017/campusconnect
depends_on:
  - mongodb
```

```
mongodb:
  image: mongo
  container_name: mongodb
  volumes:
    - student-mongo-data:/data/db
```

```
volumes:
  student-mongo-data:
```

Modify the configuration to match your actual project, port, start command, database name, and Dockerfile location.

Verification: compose.yaml defines the services and the required database volume.

Step 11 – Run the Complete Application Using Docker Compose

From the directory containing the Compose file, run:

```
docker compose up
```

For background execution, you may use:

```
docker compose up -d
```

Verify the running services:

```
docker compose ps
```

Check service logs when troubleshooting:

```
docker compose logs
```

Test the Student REST API again using Postman.

Final verification flow:

```
Postman / React → Student API Container → MongoDB Container → Docker Volume
```

Step 12 – Stop and Clean Up

Stop and remove the Compose application:

```
docker compose down
```

Do not remove the MongoDB volume when you need to demonstrate persistence. The volume should remain available for the persistence test.

4. Basic Docker Commands to Demonstrate

Command	Purpose
docker --version	Check Docker installation

docker images	List local images
docker ps	List running containers
docker ps -a	List all containers
docker stop <container>	Stop a container
docker start <container>	Start a stopped container
docker restart <container>	Restart a container
docker logs <container>	View container logs
docker network ls	List Docker networks
docker volume ls	List Docker volumes
docker compose up	Start the Compose application
docker compose ps	View Compose service status
docker compose logs	View Compose service logs
docker compose down	Stop and remove Compose containers/network

5. Suggested Project Structure

```
CampusConnect/
├── frontend/
├── backend/
│   ├── source files
│   ├── package.json / pom.xml
│   └── ...
├── Dockerfile
├── compose.yaml
├── .dockerignore
├── README.md
└── screenshots/
```

6. Submission Requirements

Submit the following:

- Existing CampusConnect / Lab 4 application.
- Dockerfile.
- compose.yaml or docker-compose.yml.
- .dockerignore.
- Updated README.md documenting the complete process.
- Any configuration files required for environment variables, without exposing passwords or secrets.

Include screenshots/evidence for the following:

22. Docker installation/version.
23. Lab 4 Student REST API running before Dockerization.
24. Dockerfile.
25. Successful student-api:v1 image build.
26. docker images output.
27. Running Student API container using docker ps.
28. Postman testing of the containerized Student REST API.
29. MongoDB container running.
30. Docker network showing the API and MongoDB containers.
31. Configuration showing MONGO_URI uses the MongoDB service/container name.
32. Environment-variable configuration.

33. MongoDB volume creation/mounting.
34. Persistence test showing data remains after MongoDB container recreation.
35. Compose configuration.
36. Successful docker compose up.
37. docker compose ps output.
38. Postman testing of the complete Compose-based application.
39. Docker/Compose logs where relevant.

7. README.md Requirements

The README should briefly document:

- Project overview and relation to Lab 4.
- Docker installation/verification commands used.
- Dockerfile explanation.
- Docker image build command.
- Container run and port mapping.
- Postman API testing.
- Docker network configuration.
- localhost vs mongodb service/container name.
- Environment variables.
- MongoDB volume and persistence test.
- Docker Compose configuration and commands.
- Troubleshooting issues encountered and how they were resolved.

8. Important Notes

- Do not rebuild the application from scratch. Continue using the Student REST API developed in Lab 4.
- Follow the steps in order and verify each stage before moving to the next.
- Use your actual Lab 4 endpoints, port, database name and technology stack where they differ from the examples.
- Do not hard-code passwords, database credentials or other secrets in the Dockerfile or Compose file.
- The Dockerfile and Compose examples are templates; modify them to match your implementation.
- The objective of this lab is to gain practical experience packaging, running, networking, configuring, persisting and orchestrating the Lab 4 application using Docker.

9. Final Lab Flow

```
Lab 4 Student REST API
  ↓
Verify API
  ↓
Create Dockerfile
  ↓
Build Docker Image
  ↓
Run Student API Container
```

↓
Test with Postman
↓
Run MongoDB Container
↓
Docker Network
↓
Environment Variables
↓
MongoDB Volume / Persistence
↓
Docker Compose
↓
Test Complete Application

End of Lab 5 Assignment